SEMESTER PROJECT REPORT

Title: Hack-Trix Cybersecurity Network Simulation
Course: Data Structures & Algorithms

Asjid Samad-2024119 Muhammad Abdullah-2024321 Warda Gul-2024663

---

1. Introduction

The purpose of this semester project was to design and implement a simplified cybersecurity simulation named **Hack-Trix**, using only fundamental C++ data-structures. The program simulates a small computer network, allows attacks, spread of vulnerabilities, maintains logs, and enables administrative operations such as undo actions, scanning, and BFS-based pathfinding.

The project demonstrates understanding of:

- Linked lists (singly and doubly)
- Circular linked structures
- Stacks and queues
- Graphs using adjacency matrices
- Breadth-First Search (BFS)

---

2. Project Objectives

1. Use core data-structures to model a network security environment.
2. Demonstrate pointer manipulation and linked-list operations.
3. Implement a graph in matrix form and use BFS for pathfinding.
4. Simulate cyber-attacks, exploit spread, and vulnerability detection.
5. Store logs and allow undo operations using stack behavior.
6. Provide a complete menu-driven interface for user interaction.

---

3. System Overview

Hack-Trix simulates a network of systems (nodes) with security states. Each system contains:

- **Name, IP, vulnerability score, patched status, compromised status**
- Each system is stored in a **doubly-linked list**, allowing efficient traversal in both directions.

Additional layers include:

- Event logs (singly linked list)
- Attack queue (circular array)
- Undo stack (linked stack)
- Circular scanner
- Graph of network connections (adjacency matrix)

This modular low-level design imitates real cybersecurity operations in a simplified academic environment.

---

4. Data Structures Used

4.1 Doubly Linked List (Systems)

Stores all systems (nodes) with fields such as ID, name, IP, vulnerability, and compromised state.

4.2 Singly Linked List (Logs)

Used for storing logs (attack logs, scanner alerts, BFS outputs). This demonstrates append-only behavior and sequential traversal.

4.3 Circular Linked List (Scanner)

Implements a continuous network scanner that loops through all systems, visiting each in turn to check their status and detects high vulnerability systems (vuln ≥ 0.8)

- Helps simulate automated monitoring tools

4.4 Stack (Undo System)

Implements LIFO behavior to restore the last system state by storing system ID and previous compromised status, allowing undo after successful or failed attacks.

4.5 Circular Queue (Attack Queue)

Array based queue used to enqueue/dequeue incoming attack requests. Fixed-size circular buffer of size 30. Displays queue, detects overflow, and processes attacks

4.6 Graph (Adjacency Matrix)

Represents the network topology of undirected edges between systems. Displays queue, detects overflow, and processes attacks. Used by BFS for shortest-path discovery

---

5. Algorithms Implemented

5.1 Attack Simulation Algorithm

Every attack attempts exploitation based on:

effective_vulnerability = vuln

if patched: effective_vulnerability *= 0.4

if effective_vulnerability >= 0.6 → COMPROMISE

else → FAIL

A deterministic (non-random) exploit method was used to produce consistent logic for testing.

5.2 Spread Algorithm

When a system is breached, the system tries to compromise all its neighbors:

1. Check adjacency list
2. For each neighbor, push undo state
3. Attempt exploit
4. Log results

This simulates lateral movement inside a compromised network.

5.3 BFS Pathfinding

The BFS function finds the shortest number of hops between two systems by IP.
Parent array reconstructs the final route, printed in IP form (e.g. 192.168.0.1 → 192.168.0.3 → 192.168.0.5).

---

6. Program Features (Menu System)

The menu includes:

1. Show all systems
2. Show logs
3. Enqueue attack
4. Show attack queue
5. Process next attack
6. Undo last action
7. Run network scanner
8. Show adjacency matrix
9. Add/Remove connections
10. Find path using BFS
11. Exit

All features are fully functional and interact with shared data structures.

---

8. Conclusion

This project successfully implemented a full cybersecurity simulation using only low-level C++ constructs such as pointers, linked lists, stacks, queues, and graphs.
Hack-Trix fulfills the semester requirements and showcases how traditional data structures can be used to build complex simulation systems.
